

Báo cáo rà soát test code

<!-- framework: Postman script | run_id: run_20260620_202817_620506 -->

Framework: Postman script | Run ID: run_20260620_202817_620506

Tóm tắt kỹ thuật

- Framework: Postman (Newman) script.
- Kết quả chạy: Tất cả các test đều pass (passed: True), không có lỗi thực thi.
- Coverage: Không hỗ trợ đo lường coverage (coverage_supported: False).
- Issue type: none – báo cáo “Newman execution passed”.
- Bản kiểm thử được xem xét: order-pricing.postman_collection.json (các test case TC-001 ... TC-005).
- Bản nguồn: Không có mã nguồn server được cung cấp, chỉ có collection Postman.
- Trình cảnh lớn nhất: Không có lỗi nghiêm trọng ngăn chạy test; tuy nhiên có một số khoảng trống về kiểm thử (thiếu kiểm tra schema, thiếu trường hợp biên giới, và thiếu kiểm tra tính toán tổng cho các kịch bản khác).

Lỗi nghiêm trọng

Không phát hiện.

Test còn thiếu

- Thiếu kiểm tra schema JSON phản hồi – yêu cầu “pm.response.to.have.jsonSchema(schema) where applicable” nhưng collection không có bất kỳ pm.response.to.have.jsonSchema nào.
- Bằng chứng: Trong mọi event.test chỉ có pm.response.to.have.status và các pm.expect riêng lẻ, không có dòng pm.response.to.have.jsonSchema.
- Ảnh hưởng: Không bảo đảm cấu trúc phản hồi luôn đúng, có thể bỏ sót lỗi thay đổi trường hoặc kiểu dữ liệu.
- Thiếu kiểm tra trường hợp biên giới “subtotal = 0” – mặc dù có TC-003 trong collection, nhưng yêu cầu trong Requirement chỉ liệt kê các trường hợp “Quote calculation for VIP domestic fragile order”, “Negative store credit”, “Missing subtotal”, “Invalid destination”, và “Discount and shipping calculations for various scenarios”. TC-003 không được mô tả trong Requirement, do đó đây là test bổ sung không được yêu cầu → không tính lỗi.
- Thiếu kiểm tra “storeCredit = 0” (giá trị hợp lệ) – không có test xác nhận rằng khi storeCredit bằng 0, tính toán tổng vẫn đúng và không trả về lỗi.
- Bằng chứng: Không có test case nào gửi storeCredit: 0 trong yêu cầu hợp lệ (TC-001 có 10, TC-005 có -1).
- Ảnh hưởng: Có thể có lỗi logic khi giá trị bằng 0 không được xử lý đúng.
- Thiếu kiểm tra “weightKg = 0” – không có test cho trọng lượng bằng 0, một giá trị biên giới quan trọng.
- Bằng chứng: Tất cả các test đều có weightKg > 0.
- Ảnh hưởng: Rủi ro lỗi khi hệ thống tính phí vận chuyển cho trọng lượng 0.

Assertion yếu

- TC-001: Kiểm tra body.total với giá trị cứng 100.5. Giá trị này được tính dựa trên logic nội bộ (subtotal 100 + shipping 22.5 - discount 12 = 110.5, sau đó có thể trừ storeCredit 10 → 100.5). Việc so sánh trực tiếp với giá trị cứng không phản ánh công thức tính và sẽ phá vỡ nếu công thức thay đổi.
- Bằng chứng: pm.expect(body.total).to.eql(100.5); trong script.
- Ảnh hưởng: Assertion yếu, không kiểm chứng công thức tổng hợp, chỉ kiểm tra kết quả cuối cùng.
- TC-002, TC-004, TC-005: Các assertion chỉ kiểm tra body.error có chứa một chuỗi. Không kiểm chứng cấu trúc lỗi (ví dụ: trường code, message) hoặc giá trị chi tiết.
- Bằng chứng: pm.expect(body.error).to.include("subtotal"); (tương tự cho các test khác).
- Ảnh hưởng: Nếu API thay đổi cách trả lỗi (ví dụ: dùng errors array), test vẫn có thể pass mà không phát hiện lỗi.

Rủi ro maintainability

- Hard-coded giá trị trong các assertion (ví dụ 100.5, 12, 22.5, 110.5) làm cho collection khó bảo trì khi quy tắc kinh doanh thay đổi.
- Không có reuse của biến hoặc hàm helper để tính toán giá trị mong đợi; mỗi test lặp lại cùng các giá trị.
- Không có môi trường cấu hình cho schema hoặc các hằng số (discount rate, shipping base) → khi thay đổi, phải sửa nhiều test đồng thời, dễ gây lỗi.

Gợi ý sửa ngay

1. Thêm kiểm tra schema JSON cho các phản hồi thành công và lỗi:

```
const schema = {
  "type": "object",
  "properties": {
    "total": {"type": "number"},
    "diagnostics": {
      "type": "object",
      "properties": {
        "discount": {"type": "number"},
        "shipping": {"type": "number"},
        "totalBeforeCredit": {"type": "number"}
      },
      "required": ["discount", "shipping", "totalBeforeCredit"]
    },
    "error": {"type": "string"}
  },
  "required": ["total", "diagnostics"]
};

pm.test("response matches schema", function () {
  pm.response.to.have.jsonSchema(schema);
});
```

2. Biến đổi các assertion tính toán để dựa trên công thức thay vì giá trị cứng:

```
const body = pm.response.json();
const expectedTotal = body.diagnostics.totalBeforeCredit - body.storeCredit;
```

```
pm.expect(body.total).to.eql(expectedTotal);
```

3. Thêm test case cho `storeCredit = 0` và `weightKg = 0` để bao phủ các biên giới.

4. Tách hàng số (discount rate, shipping base) vào biến môi trường Postman (`pm.environment.get("DISCOUNT_RATE")...`) để dễ cập nhật.

5. Mở rộng kiểm tra lỗi: xác nhận cấu trúc lỗi (ví dụ `pm.expect(body).to.have.property('error');` `pm.expect(body.error).to.be.an('object');`) và kiểm tra mã lỗi cụ thể nếu có.
